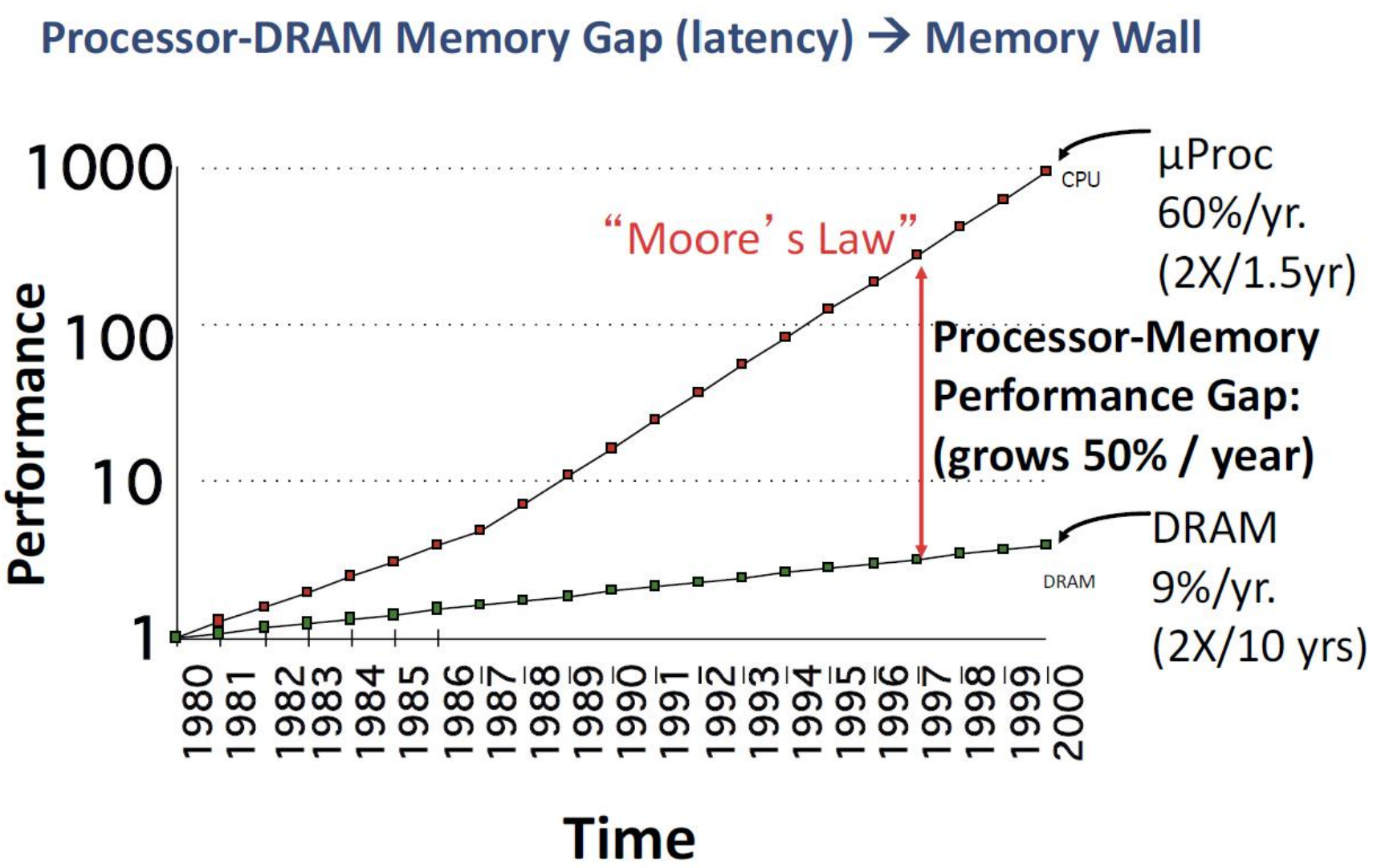
Cache coherence in shared memory system

Cache coherence means keeping the copies of the same data in different CPU caches consistent with each other.

Memory until now …

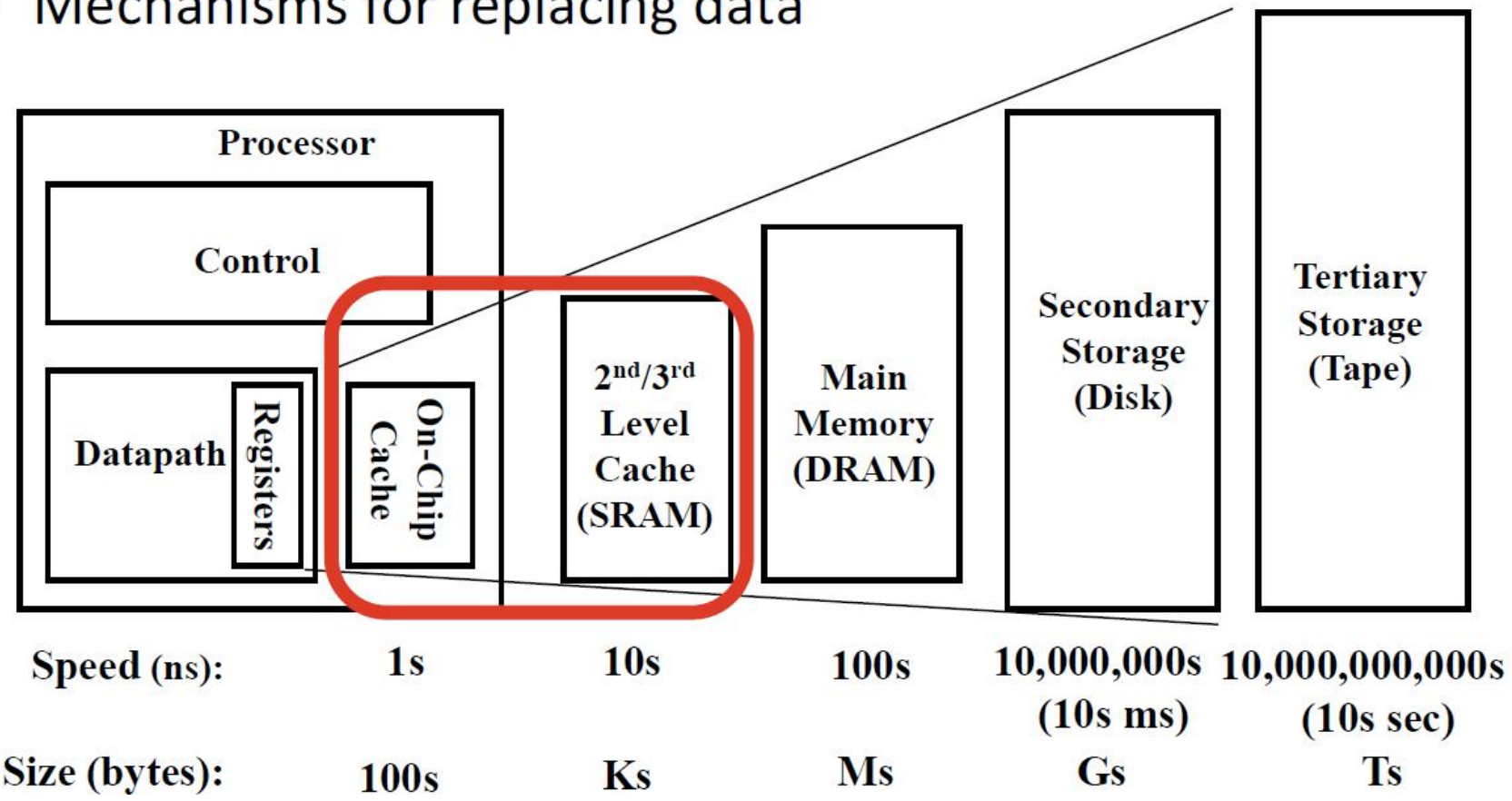
- We've relied on a very simple model of memory for most this class
  - Main Memory is a linear array of bytes that can be accessed given a memory address
  - Also used registers to store values
- Reality is more complex. There is an entire memory system.
  - Different memories exist at different levels of the computer
  - Each vary in their speed, size, and cost

Why Cares About the Memory Hierarchy?



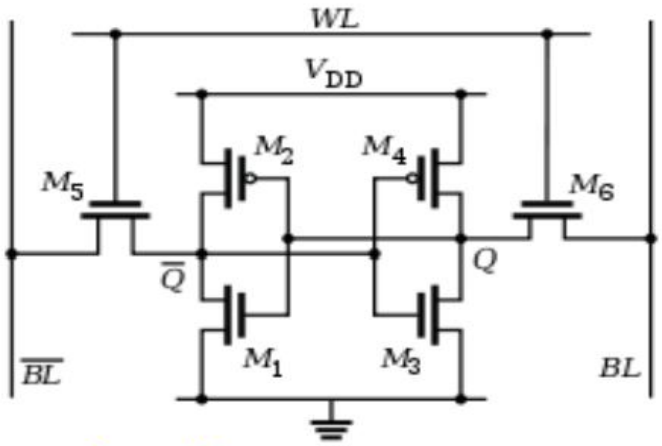
Architecture Approach: Memory Hierarchy

- Keep **most recent accessed data** and **its adjacent data** in the smaller/faster caches that are closer to processor
- Mechanisms for replacing data



Random-Access Memory (RAM)

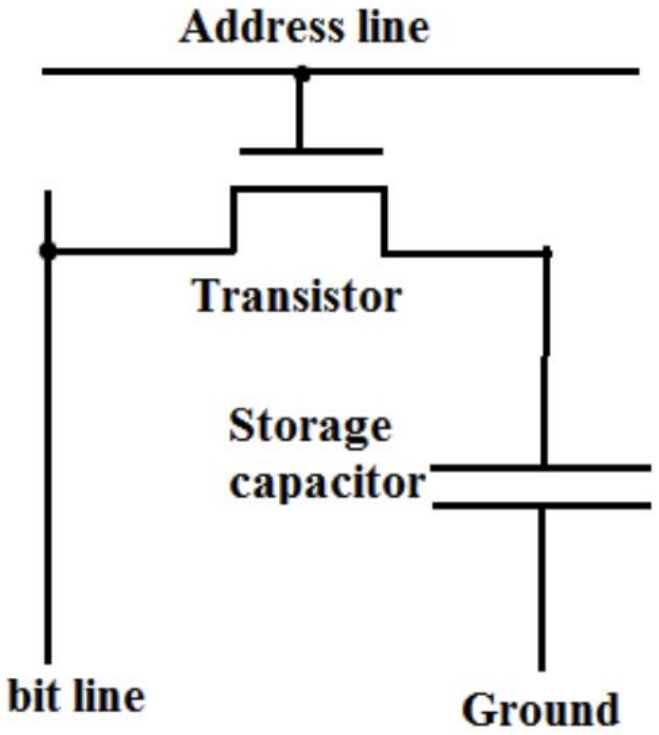
- Key features
  - RAM is packaged as a chip.
  - Basic storage unit is a cell (one bit per cell).
  - Multiple RAM chips form a memory.



- **Static RAM (SRAM)**
  - Each cell stores bit with a six-transistor circuit.
  - Retains value indefinitely, as long as it is kept powered.
  - Relatively insensitive to disturbances such as electrical noise.
  - Faster and more expensive than DRAM.

Random-Access Memory (RAM)

- **Dynamic RAM (DRAM)**
  - Each cell stores bit with a capacitor and transistor.
  - Value must be refreshed every 10-100 ms.
  - Sensitive to disturbances.
  - Slower and cheaper than SRAM.



How to Exploit Memory Hierarchy: Program Behavior: Principle of Locality

Temporal Locality (Time-based Reuse): If a program uses some data or instruction now, it is likely to use the same data or instruction again very soon.

```
for(int i = 0; i < 100; i++) {
    sum = sum + 5;
}
```

Here, the variable ***sum*** is used repeatedly in every loop iteration.

So the processor keeps ***sum*** in cache memory for quick access.

How to Exploit Memory Hierarchy: Program Behavior: Principle of Locality

Spatial Locality (Nearby Access): If a program accesses one memory location, it is likely to access nearby memory locations soon.

```
int arr[5] = {1, 2, 3, 4, 5};

for(int I = 0; I < 5; i++) {
    printf("%d", arr[i]);
}
```

When *arr[0]* is accessed, the processor expects *arr[1], arr[2],* etc. will also be accessed because they are stored next to each other in memory.

Sources of Locality

- Temporal locality
  - Code within a loop
  - Same instructions fetched repeatedly
- Spatial locality
  - Data arrays
  - Local variables in stack
  - Data allocated in chunks (contiguous bytes)

Writing Cache Friendly Code

- Repeated references to variables are good (temporal locality)
- Stride-1 reference patterns are good (spatial locality)
- Examples:
  - cold cache, 4-byte words, 4-word cache blocks

```
int sumarrayrows(int a[M][N])
{
    int i, j, sum = 0;

    for (i = 0; i < M; i++)
        for (j = 0; j < N; j++)
            sum += a[i][j];
    return sum;
}
```

Miss rate = 1/4 = 25%

```
int sumarraycols(int a[M][N])
{
    int i, j, sum = 0;

    for (j = 0; j < N; j++)
        for (i = 0; i < M; i++)
            sum += a[i][j];
    return sum;
}
```

Miss rate = 100%

Writing Cache Friendly Code

Suppose memory looks like this:

a[0][0] a[0][1] a[0][2] a[0][3] a[0][4] ...

When CPU accesses: a[0][0]

Cache loads: a[0][0], a[0][1], a[0][2], a[0][3]

What Happens Next?

Access	*Hit or Miss*	*Reason*
a[0][0]	Miss	Block loaded first time
a[0][1]	Hit	Already in cache
a[0][2]	Hit	Already in cache
a[0][3]	Hit	Already in cache

```
int sumarrayrows(int a[M][N])
{
    int i, j, sum = 0;

    for (i = 0; i < M; i++)
        for (j = 0; j < N; j++)
            sum += a[i][j];
    return sum;
}
```

Miss rate = 1/4 = 25%

Then next block starts:

Access	*Hit or Miss*
a[0][4]	Miss
a[0][5]	Hit
a[0][6]	Hit
a[0][7]	Hit

Miss Rate Calculation

For every 4 accesses:

1 miss, 3 hits

So: Miss Rate=1/4=25%.

Writing Cache Friendly Code

```
int sumarraycols(int a[M][N])
{
    int i, j, sum = 0;

    for (j = 0; j < N; j++)
        for (i = 0; i < M; i++)
            sum += a[i][j];
    return sum;
}
```

Miss rate = 100%

Access	***Result***
a[0][0]	Miss
a[1][0]	Miss
a[2][0]	Miss
a[3][0]	Miss

In C, arrays are stored in **row-major order**.

a[0][0] a[0][1] a[0][2] a[0][3] a[1][0] a[1][1] ...

Rows are contiguous, not columns.

When CPU accesses: a[0][0]
The cache loads: a[0][0], a[0][1], a[0][2], a[0][3]
But the next access is: a[1][0]
which is not inside the current cache block.
So another new block must be loaded.
Again next access: a[2][0] needs another block.

Every access causes a cache miss.
Total accesses: Every access is a miss
Miss Rate=All Misses/Total Accesses=1=100%

Matrix Multiplication Example

- Major cache effects to consider
  - Total cache size
    - Exploit temporal locality and blocking)
  - Block size
    - Exploit spatial locality

Exploit = Take full advantages

dividing the big matrix into smaller blocks (or tiles).

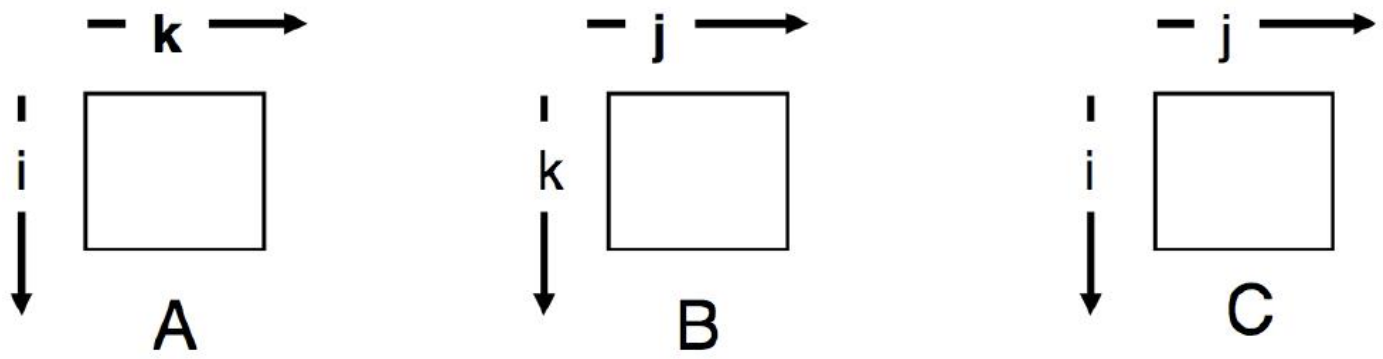
- Description:
  - Multiply N x N matrices
  - $O(N^3)$ total operations
  - Accesses
    - N reads per source element
    - N values summed per destination
      - but may be able to hold in register

```
/* ijk */
for (i=0; i<n; i++)    {
  for (j=0; j<n; j++) {
    sum = 0.0;
    for (k=0; k<n; k++)
      sum += a[i][k] * b[k][j];
    c[i][j] = sum;
  }
}
```

*Variable **sum** held in register*

Miss Rate Analysis for Matrix Multiply

- Assume:
  - Cache line size = 32 Bytes (big enough for 4 64-bit words)
  - Matrix dimension (N) is very large
    - Approximate 1/N as 0.0
  - Cache is not even big enough to hold multiple rows
- Analysis method:
  - Look at access pattern of inner loop

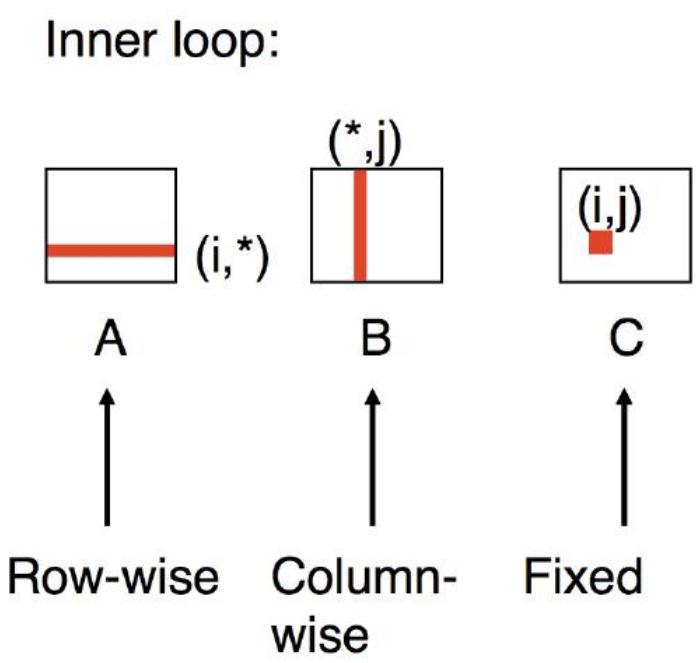


Layout of C Arrays in Memory (review)

- C arrays allocated in row-major order
  - each row in contiguous memory locations
- Stepping through columns in one row:
  - for(i = 0; i < N; i++)
    sum += a[0][i];
  - accesses successive elements
  - if block size (B) > 4 bytes, exploit spatial locality
    - compulsory miss rate = 4 bytes / B
- Stepping through rows in one column:
  - for(i = 0; i < n; i++)
    sum += a[i][0];
  - accesses distant elements
  - no spatial locality!
    - compulsory miss rate = 1 (i.e. 100%)

Matrix Multiplication (ijk)

```
/* ijk */
for (i=0; i<n; i++)  {
  for (j=0; j<n; j++) {
    sum = 0.0;
    for (k=0; k<n; k++)
      sum += a[i][k] * b[k][j];
    c[i][j] = sum;
  }
}
```

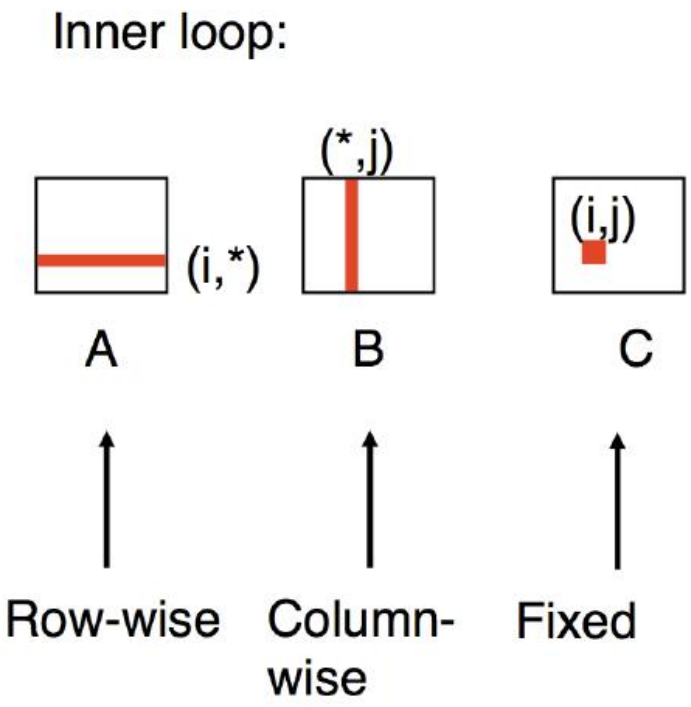


- Misses per Inner Loop Iteration:

A	B	C
0.25	1.0	0.0

Matrix Multiplication (jik)

```
/* jik */
for (j=0; j<n; j++) {
  for (i=0; i<n; i++) {
    sum = 0.0;
    for (k=0; k<n; k++)
      sum += a[i][k] * b[k][j];
    c[i][j] = sum
  }
}
```

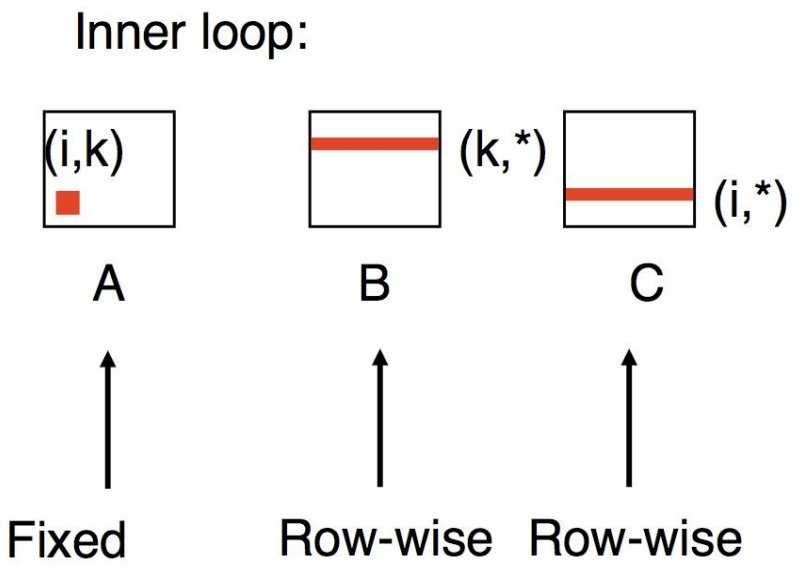


- Misses per Inner Loop Iteration:

A	B	C
0.25	1.0	0.0

Matrix Multiplication (kij)

```
/* kij */
for (k=0; k<n; k++) {
  for (i=0; i<n; i++) {
    r = a[i][k];
    for (j=0; j<n; j++)
      c[i][j] += r * b[k][j];
  }
}
```

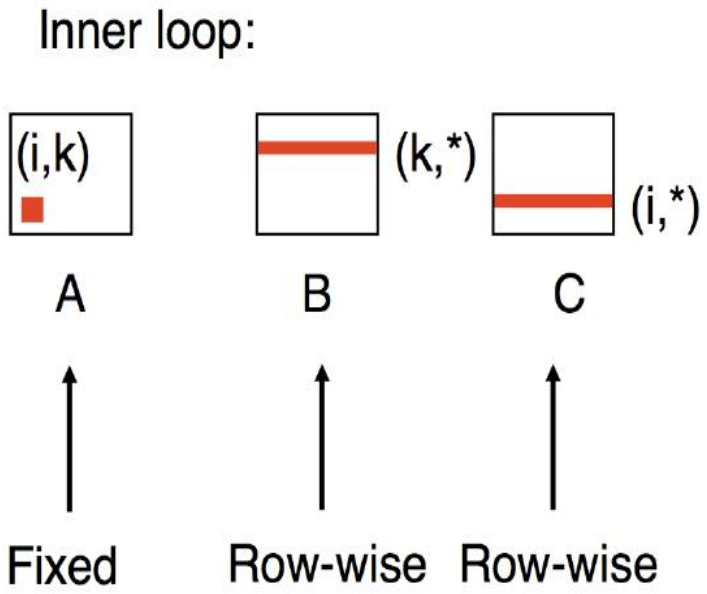


- Misses per Inner Loop Iteration:

A	B	C
0.0	0.25	0.25

Matrix Multiplication (ikj)

```
/* ikj */
for (i=0; i<n; i++) {
  for (k=0; k<n; k++) {
    r = a[i][k];
    for (j=0; j<n; j++)
      c[i][j] += r * b[k][j];
  }
}
```

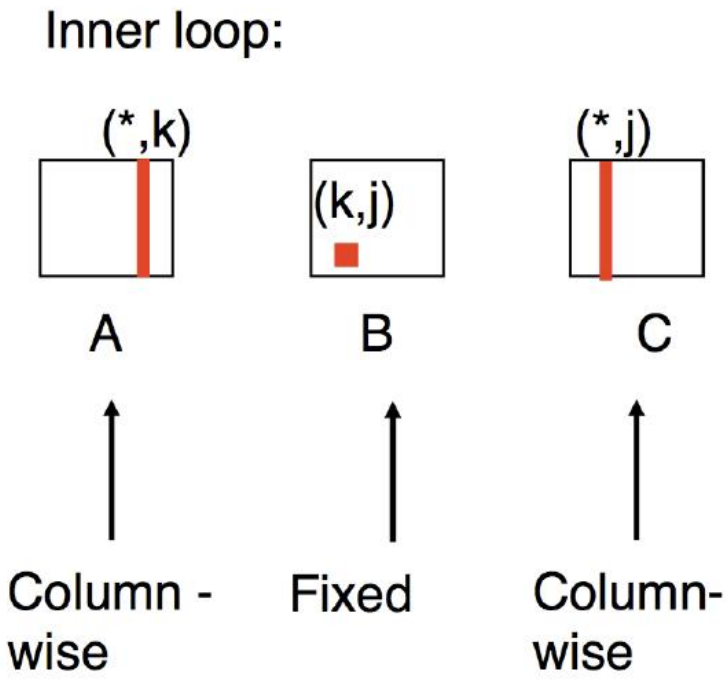


- Misses per Inner Loop Iteration:

A	B	C
0.0	0.25	0.25

Matrix Multiplication (jki)

```
/* jki */
for (j=0; j<n; j++) {
  for (k=0; k<n; k++) {
    r = b[k][j];
    for (i=0; i<n; i++)
      c[i][j] += a[i][k] * r;
  }
}
```

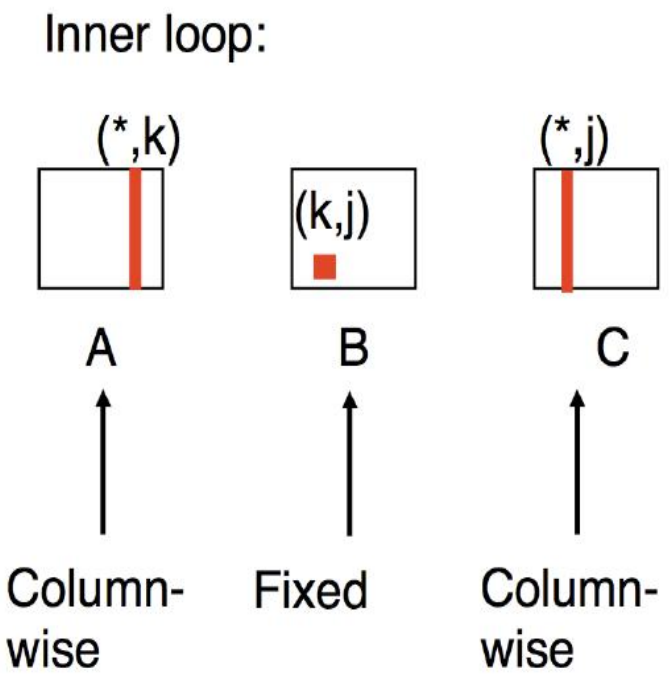


- Misses per Inner Loop Iteration:

A	B	C
1.0	0.0	1.0

Matrix Multiplication (kji)

```
/* kji */
for (k=0; k<n; k++) {
  for (j=0; j<n; j++) {
    r = b[k][j];
    for (i=0; i<n; i++)
      c[i][j] += a[i][k] * r;
  }
}
```



- Misses per Inner Loop Iteration:

A	B	C
1.0	0.0	1.0

Summary of Matrix Multiplication

ijk (& jik):
- 2 loads, 0 stores
- misses/iter = **1.25**

```
for (i=0; i<n; i++)  {
  for (j=0; j<n; j++) {
    sum = 0.0;
    for (k=0; k<n; k++)
      sum += a[i][k] *
    b[k][j]; c[i][j] = sum;
  }
}
```

kij (& ikj):
- 2 loads, 1 store
- misses/iter = **0.5**

```
for (k=0; k<n; k++) {
  for (i=0; i<n; i++) {
    r = a[i][k];
    for (j=0; j<n; j++)
      c[i][j] += r * b[k][j];
  }
}
```

jki (& kji):
- 2 loads, 1 store
- misses/iter = **2.0**

```
for (j=0; j<n; j++) {
  for (k=0; k<n; k++) {
    r = b[k][j];
    for (i=0; i<n; i++)
      c[i][j] += a[i][k] * r;
  }
}
```